ARI 410/510 Lab 3:
Multimodal Classification with Pretrained Encoders

University of Michigan-Flint

Fall 2026

(See course Canvas page for due date. Post questions on Discord in the #homework-help channel.)

1 Overview

In this lab, you will build classifiers that use text only, images only, and both together (*multimodal fusion*). You will work with the **Hateful Memes** dataset from Facebook AI, where each example is a meme (an image with overlaid text) labeled as *hateful* or *not hateful*. Rather than training image and text models from scratch, you will use pretrained models (BERT for text, ResNet for images) as *encoders* that turn each input into a vector, and train small neural networks on top of those vectors. This is a very common pattern in practice, and the same approach works for audio, video, and other kinds of data. It is also a reasonable first model for some of the competitions in Phase 2b of the project: if another team's task has text, images, or both, the pipeline you build here is a good place to start. So build it in a way you can reuse.

This dataset was built specifically to make single-modality models struggle: it includes "confounder" memes where the same text is harmless with one image and hateful with another, and vice versa. So it is a good test of whether combining modalities actually helps. You are *not* expected to find the best possible model. The goal is to apply simple approaches carefully, follow good experimental practice, and understand what you see.

Content note. As the name suggests, this dataset contains offensive and hateful content, including slurs and attacks on people based on race, religion, gender, disability, and other characteristics. You will mostly be working with vectors and metrics, but you will look at some examples. If this is a problem for you, contact the instructor.

Tips: read this before you start:

- **Keep a short work log** as you go: a few lines each time you work on the lab about what you tried, what broke, and what you figured out. It takes two minutes per session and makes the reflection much easier to write.
- If you use any generative AI, you will fill out a structured statement about how you used it (Section 11). Attaching your chat logs is optional but encouraged; if you plan to do that, set up whatever you need to export them *before* you begin.
- The images make this dataset large. Develop your code on a small subset first (say, 200 examples) and run on everything only once it works. **Compute your embeddings only once.** The starter notebook (see Canvase) saves them to your Google Drive, and from then on each new Colab session can load them in seconds without downloading the images again (see Section 12).

Starter notebook and code snippets. The starter notebook (on Canvas) gives you the building blocks: it downloads and loads the data, includes a helper for viewing a meme, and sets up both pretrained encoders (BERT for text, ResNet-18 for images). Running it produces text and image vectors for every meme, saved to your Google Drive. Building, training, and comparing the classifiers is up to you. The snippets below are not a complete solution and are provided "as is." You do not need to use them.

2 Getting the Data

The dataset is available on Hugging Face and uses Git Large File Storage (LFS):

```
git lfs install
git clone https://huggingface.co/datasets/neuralcatcher/hateful_memes
```

Each split is a `.jsonl` file with one meme per line: an `id`, the path to the image (`img`), the meme's `text`, and its `label` (1 = hateful, 0 = not hateful). The images are in the `img/` folder. Use `train.jsonl` for training, `dev_seen.jsonl` as your development set, and `test_seen.jsonl` as your test set. You can ignore the `unseen` files.

3 Part 0: Look at the Data First

Before you train anything, get to know the data. This is the first step of any good neural network training process, and it is easy to skip. Do all of the following before Part 1:

1. **Label distribution.** How many memes are in each split, and what fraction are hateful? Make a plot. Is the hateful fraction the same in every split? It isn't, and that matters for how you read your results later.
2. **Look at examples.** Look through at least 10 memes from the training set (the starter notebook's `show_meme` helper makes this quick). Write down a few things you notice: patterns in the text or images, what the hateful ones have in common, anything surprising or messy (blurry images, text that doesn't match the image, and so on).
3. **Be the model.** Pick 10 memes from the dev set and, *without looking at the labels*, decide for each one whether it is hateful. Then check the real labels and compute your own accuracy. Which ones did you get wrong, and why? Did you need both the text and the image to decide? Keep your number: it is a human baseline to compare your models against.

4 Part 1: Text-Only Classification

4.1 Step 1: Text embeddings

The starter notebook uses a pretrained language model (BERT) to turn each meme's text into a single vector of 768 numbers: the hidden state of the special `[CLS]` token from BERT's last layer. BERT is used as-is: it is never trained on memes, and we only keep its vectors. Run your code on a small subset first to make sure it works, then on everything.

4.2 Step 2: The classifier

Build a small feedforward neural network that takes an embedding and outputs a single number (a logit) for "hateful", for example,:

```
import torch.nn as nn

class Classifier(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, 1)  # one output: the logit for "hateful"

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

model = Classifier(input_dim=768, hidden_dim=TODO)
```

4.3 Step 3: Training and evaluation

Train with binary cross-entropy loss, using mini-batches, and evaluate on the **dev set** with accuracy, precision, recall, and F1-score. Use the dev set for every decision you make (hidden size, number of epochs, learning rate, and so on).

Sanity checks first. Before training your text model for real, do these quick checks. They catch most bugs early, and they are standard practice.

1. **A dumb baseline.** Compute all metrics on the dev set for a "model" that always predicts *not hateful*. Any real model needs to beat this.
2. **Loss at the start.** Before any training, compute the loss on a batch. A freshly initialized network outputs logits near 0 (probabilities near 0.5), so the loss should be close to $\ln 2 \approx 0.69$. If it is *far* from that, something is wrong.
3. **Overfit a tiny batch.** Train on just 20 examples for many epochs. The training loss should go to nearly 0 and training accuracy to 100%. If your model can't memorize 20 examples, there is a bug in your training loop.
4. **Watch the curves.** When you train on the full training set, record the training loss and the dev loss after every epoch and plot both. Use the plot to choose the number of epochs: dev loss that starts going up while training loss keeps going down means the model is overfitting.

For the models in Parts 2 and 3, you only need check 4 (a training/dev loss plot for each). Tip: write one function that takes training and dev vectors and labels, trains a classifier, and returns the dev metrics and loss curves. You will use it for every model in this lab, and maybe again in Phase 2b of the project.

```
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score

criterion = nn.BCEWithLogitsLoss()  # applies the sigmoid inside the loss
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# e.g., train_X = X_text['train'] (shape [n_train, 768]); train_y = y['train'] (0 or 1)
loader = DataLoader(TensorDataset(train_X, train_y.float().unsqueeze(1)), batch_size=64,
    shuffle=True)

for epoch in range(TODO):
    model.train()
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()

    # evaluate on the dev set after each epoch
    model.eval()
    with torch.no_grad():
        dev_probs = torch.sigmoid(model(dev_X)).squeeze(1)
    dev_preds = (dev_probs >= 0.5).int()
    print(epoch, f1_score(dev_y, dev_preds), roc_auc_score(dev_y, dev_probs))
```

Save your trained model so you can run it on the test set later.

5 Part 2: Image-Only Classification

5.1 Step 1: Image embeddings

The starter notebook also sets up a pretrained image model, ResNet-18, as an encoder. ResNet-18 was trained to sort photos into 1,000 ImageNet categories (dogs, cars, mushrooms, and so on). The notebook

removes its final classification layer, so instead of scores for those 1,000 categories it outputs the 512 numbers that layer would have used: one vector per image. Read through that part of the notebook. In your report, explain in a sentence or two why it makes sense to remove the final layer here.

5.2 Steps 2 and 3: Classifier, training, and evaluation

Train and evaluate a classifier the same way as in Section 4.2, with an input size of 512.

6 Part 3: Multimodal Fusion

Note: ARI 510 students, read ahead to the "Track-Specific Work" section before you do this step.

6.1 Early fusion (combine the features)

Concatenate each meme's text and image embeddings into one vector and train a new classifier on it:

```
combined = torch.cat((text_embeddings, image_embeddings), dim=1)  # [n, 768 + 512]
```

6.2 Simple late fusion (combine the predictions)

Use your already-trained text-only and image-only classifiers, and combine their predicted probabilities, for example by averaging them:

```
with torch.no_grad():
    p_text = torch.sigmoid(text_model(dev_text_X))
    p_image = torch.sigmoid(image_model(dev_image_X))
p_late = (p_text + p_image) / 2
```

No new training is needed for this simple version.

7 Part 4: Final Comparison

Once you have made all your choices on the dev set, evaluate all four models **once** on the test set: text-only, image-only, early fusion, and late fusion.

Because most memes in the training set are not hateful, a model can learn to lean toward predicting "not hateful," and the dev and test sets don't have the same balance as the training set. Report metrics, but think about which ones you trust the most.

8 Track-Specific Work

ARI 410: Look at the memes

Find memes in the dev set where your fusion model and the single-modality models disagree:

- at least two where early or late fusion was right but the text-only or image-only model was wrong, and
- at least one where fusion was wrong.

Look at each meme (image and text together). In your report, give each one's `id` and describe what makes it hard for a single modality, or what the fusion model may have missed. You do not need to reproduce the meme in your report; a description is enough.

ARI 510: Design a better fusion

1. **Hypothesis.** Before running Part 3, write down which modality you expect to be more useful on its own, and whether (and by how much) you expect fusion to help. Explain your reasoning.

2. **One design change.** Propose and test one change to how the two modalities are combined, and justify it. Examples: a weighted average for late fusion with the weight chosen on the dev set; training the two classifiers jointly through a combined loss; a different encoder for one modality (for example, CLIP, which was trained on images and text together). Compare it to your baseline fusion models on the dev set, then include it in your final test-set comparison.

Students in either section are welcome to also do the other section's part.

9 Deliverables

Submit:

1. your written report (PDF);
2. your code. A Jupyter notebook (`.ipynb`) is fine, as is a `.py` file or a link to a GitHub repository.

Results checklist

Make sure your report includes all of the following:

- ☐ Your Part 0 findings: the label-distribution plot, your notes on the examples, and your own accuracy on 20 dev memes.
- ☐ Your sanity checks: the starting loss and the tiny-batch result for the text model, and a training/dev loss plot for each model.
- ☐ A **dev-set** results table: all metrics for the always-"not hateful" baseline and all four models (plus your design change, for 510).
- ☐ A **test-set** results table: all metrics for the same models, computed once, after all choices were made on the dev set.

10 Report

Most of your grade is the report's reasoning and completion, not the code.

Everyone

1. **Looking at the data:** your Part 0 findings. What did you learn about the task before training anything, and how hard was it for you as a human?
2. **Approach:** briefly, what you did: the encoders, your classifier settings and how you chose them on the dev set (refer to your loss curves), your sanity-check results, and anything you did differently from the instructions.
3. **Results:** the dev-set and test-set tables from the checklist above.
4. **Discussion:** What do your results say about how useful the text and the images are for this task? Did fusion help, and did early or late fusion work better? Why do you think so, given how the dataset was built? How do your models compare to the dumb baseline and to your own accuracy as a human? Neither BERT nor ResNet was ever trained on memes or hate speech; what does it tell you that a small classifier on their vectors does as well as it does (or doesn't)? Which metrics did you rely on, and why?
5. **Reflection:** see below.
6. **Generative AI Use Statement:** see Section 11.

ARI 410 additionally

Your "Look at the memes" analysis.

ARI 510 additionally

Your hypothesis (stated before your results, and revisited in the Discussion) and your design change, with its justification and results.

The reflection

The reflection is about *your experience* doing this lab, not a summary of what multimodal learning is. We already know what the lab covers; we want to hear what actually happened when you did it. Use your work log. Write about specific things, for example:

- Something that didn't work, what you thought the problem was, what it actually turned out to be, and how you found out.
- A result that surprised you, and what you checked to make sure it wasn't a bug.
- A decision you made (a setting, a shortcut, a workaround), why you made it, and whether you'd make it again.
- What you would do differently if you started over.

Be concrete: name the error message, the line of code, the number that looked wrong. For example, "my dev F1 was exactly 0 for the first three epochs; I assumed the learning rate was too low, but it turned out I was predicting with a threshold on the raw logits instead of the probabilities" is a good reflection sentence. "I learned that multimodal fusion can improve performance" is not. Generic statements like that will not get credit.

11 Generative AI Use Statement (required)

Every submission must include **one** of the following.

Option A: No AI. If you used no generative AI, state exactly: *"No generative AI was used during this assignment."* You are done with this section. Please note that typing a query into Google and getting the automatic Gemini response (or something similar to that, which I will call "unintentional AI use") doesn't need to be documented here. Only include cases where you actively used a GenAI tool to help in some way (including autocomplete for coding, ChatGPT chat interface, agentic assistant like Claude Code, etc.).

Option B: Structured statement. If you used any generative AI tool you deliberately turned to (ChatGPT, Claude, Copilot, Gemini as a chatbot, etc.), complete the table below with one row per phase of your work. The point is not to confess to using AI (it is allowed) but to show *where the thinking was*: research on knowledge work finds that when people use AI, the real effort shifts from producing an artifact to verifying, integrating, and deciding whether to trust the output.[1]

For each phase, note: (1) whether you used AI, and if the answer is "yes", fill in: (2) *how* which mode(s) (see below), and what you actually asked (feel free to make this cell of the table large to explain) and (3) what you did to check, correct, or override the output. The three "modes":

- **Creation**: had the AI generate something from scratch (code, text, an idea).
- **Information**: I used it to explain, find, or summarize (e.g., "what does `BCEWithLogitsLoss` do?").
- **Advice / feedback**: I used it to improve, check, or critique something you already had (e.g., "review this function," "is AUC-ROC the right metric here?").

[1] Structure adapted from H.-P. Lee, A. Sarkar, L. Tankelevitch, I. Drosos, S. Rintel, R. Banks, & N. Wilson, "The Impact of Generative AI on Critical Thinking," CHI 2025.

Phase	**Used AI?**	**How (mode + what you asked)**	**How you checked / corrected / overrode it**
Planning & understanding			
Coding and Debugging			
Analysis & interpretation			
Report writing, formatting, editing			

Note: the Reflection must be your own writing about your own experience; do not use AI to write it.

[**Optional, but encouraged**] **Attach your logs.** For any deliberate chatbot or AI agent use, export and attach your chat transcript or coding session, or include a shared link. Set this up *before* you start (this is why it's flagged in the Overview). If you used an inline tool that cannot export a clean log (e.g., editor autocomplete), just describe how you used it instead.

12 Computational Resources

The expensive part of this lab is running BERT and ResNet once over all of the memes to produce embeddings. The starter notebook does that once and saves the results to Google Drive, and everything after that (training the small classifiers) runs quickly on a laptop. Google Colab's free tier, with a GPU runtime (*Runtime* → *Change runtime type*), is enough for the embedding step.

Once your full embeddings are saved, start each new Colab session at the notebook's "Starting a new session" section, which loads them from Google Drive: no dataset download and no GPU needed. You only need the images again to look at memes. If Colab's free GPU is busy or your session disconnects while computing embeddings, rerun the notebook: anything already saved is loaded instead of recomputed.

You may also use other computational resources that you have access to instead of Colab.

13 Other questions?

Please feel free to ask on Discord in the #homework-help channel.